

Introduction to Computer Simulation

Home Work 6

Guilherme Schewtschik (3748052)

December 2, 2025

Question 1

The following code was used for the graphing:

```
def get_Random_state(X):
    state = np.random.random(size = X)
    return np.ceil(state*2-1)*2-1

def get_Energy_state_ND(state,J,h=0):
    D = int((np.array(np.shape(state))/np.shape(state)[0]).sum())
    kern = nd.generate_binary_structure(D,1)
    idx = tuple([1] * D)
    kern[idx] = False
    arr = nd.convolve(state, kern, mode='wrap')

    return (-J*state*arr).sum()/2

def time_Evolution_Metropolis(state,J,h,beta,iterations, get_Energy_state):
    state_mu = state.copy()
    state_evolution = np.zeros((iterations, len(state)))
    net_Energy = np.zeros(iterations)
    net_Energy[0] = get_Energy_state(state,J,h)

    for i in range(0,iterations):
        state_nu = state_mu.copy()
        x = np.random.randint(0,len(state))
        state_nu[x] = state_nu[x]*-1

        E_nu = get_Energy_state(state_nu,J,h)
        E_mu = get_Energy_state(state_mu,J,h)

        if (E_nu <= E_mu)or(np.random.random()<=np.exp(-beta*(E_nu-E_mu))):
            state_mu = state_nu
            net_Energy[i] = E_nu
        else:
            net_Energy[i] = E_mu

        state_evolution = np.concatenate((state_evolution, state_mu), axis=0)

    return state_evolution, net_Energy, get_Energy_state(state_evolution[-1],J,h)

N = 1000

L = np.array([[4,4],[8,8],[16,16]])
it = np.arange(N)
E_f = np.zeros(N)

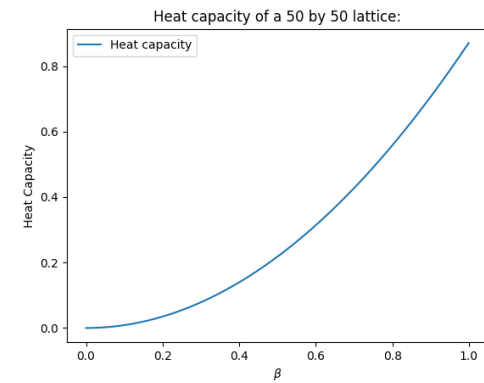
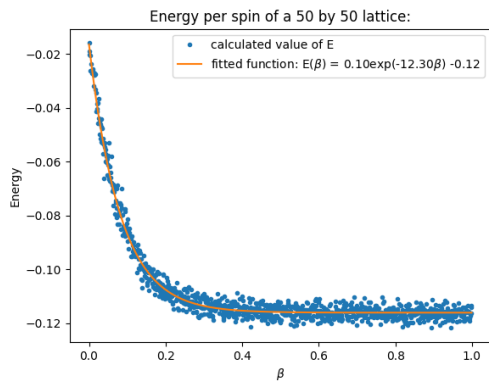
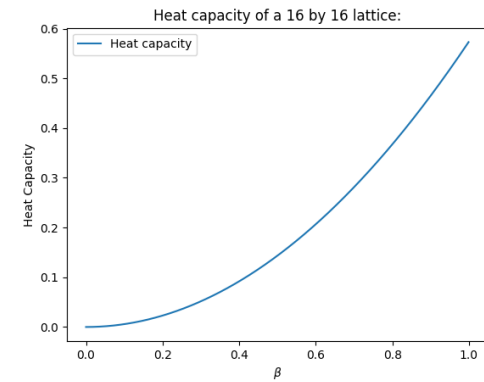
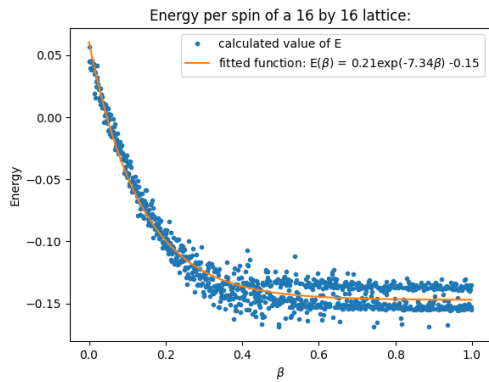
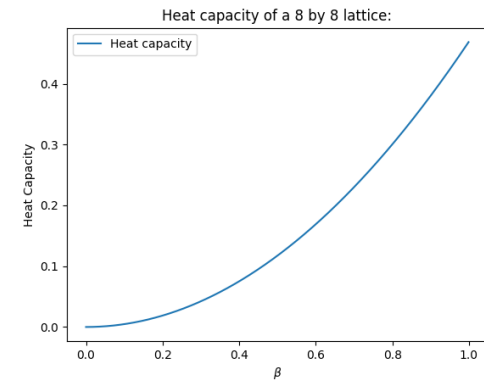
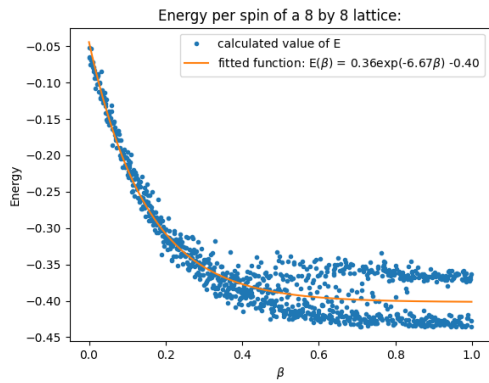
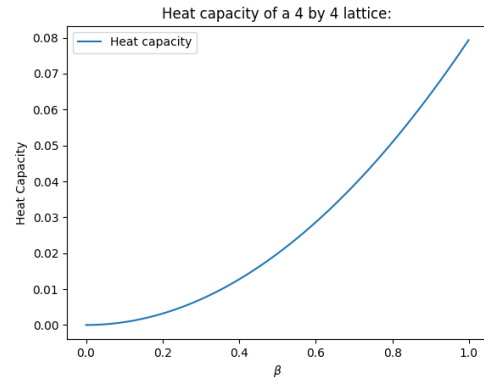
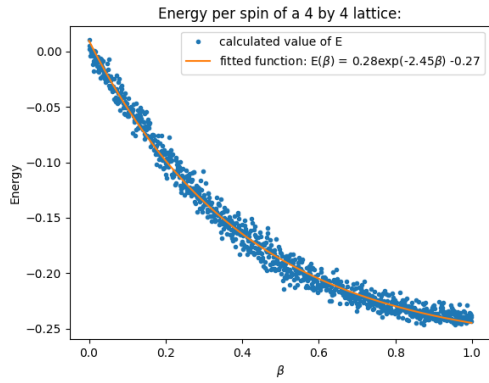
for j in tqdm(range(2,3), desc=f"E"):
    state = get_Random_state(L[j])

    for i in range(0,N):
        _, E_M, _ = time_Evolution_Metropolis(state,1,0, 0+i/1000, N, get_Energy_state_ND)
```

```
E_f[i] = get_running_average(E_M)[-1]
```

We can calculate the specific Heat capacity through:

```
C = (beta**2)*(np.mean(E[i]**2)-(np.mean(E[i]))**2)/(sizes[i,0]**2)
```



Question 2

The following code was used for question 2:

```
it = 10000

r_ee_array = np.zeros(it)
r_gy_array = np.zeros(it)

for i in tqdm(range(0,it), desc=f"Random Walks"):
    N = 75

    phi = 2*np.pi*np.random.random(size = N)
    b = np.array([np.cos(phi),np.sin(phi)]).reshape(2,N)

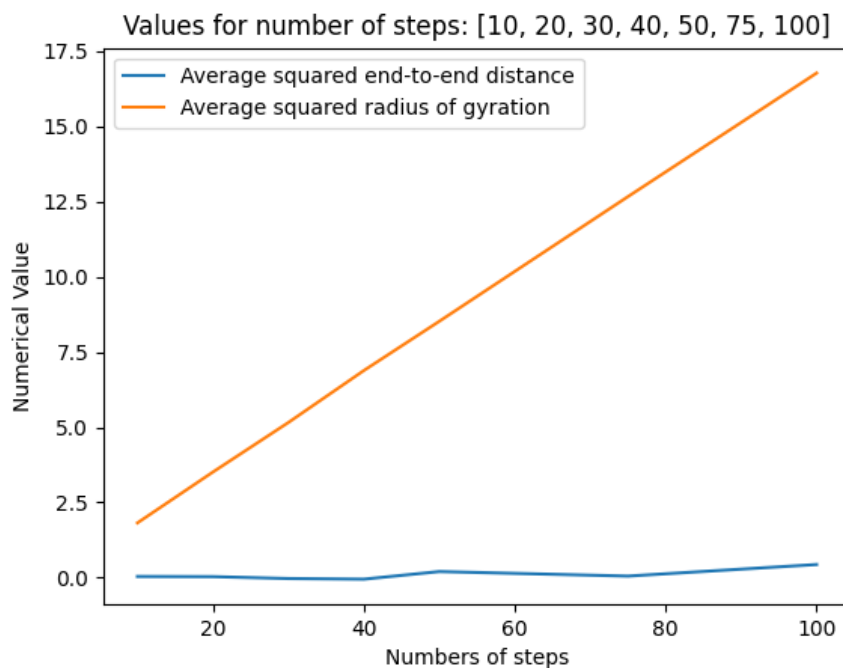
    r_temp = np.cumsum(b, axis=1)
    r = np.array([np.insert(r_temp[0,:],0, 0),np.insert(r_temp[1,:],0, 0)])

    r_com = np.mean(r,axis=1)
    r_ee = np.mean(r[0,:]*r[1,:])

    r_gy = np.mean((r[0,:] - r_com[0])**2 + (r[1,:] - r_com[1])**2)

    r_ee_array[i] = r_ee
    r_gy_array[i] = r_gy
```

With this we get the following graph:



We can clearly see that the angle of gyration scales linearly with the number of steps, while the end to end distance stays around zero no matter the number of steps.